Module: Recurrent Neural Network (RNN) and Long Short-Term Memory (LSTM)

1. Recurrent Neural Network (RNN)

What is RNN?

A Recurrent Neural Network (RNN) is a type of neural network designed to process sequential data. Unlike a basic feed-forward Artificial Neural Network (ANN), an RNN can use information from previous time steps while processing the current input.

RNNs are useful when the order of data matters.

Why do we need RNN?

A basic ANN processes inputs without inherently maintaining information from previous time steps. However, sequential data often depends on its context.

For example, in the sentence:

"I love machine learning."

Understanding a word can depend on the words that came before it.

RNN addresses this by maintaining a hidden state that carries information from previous time steps.

How does RNN work?

An RNN processes a sequence one element at a time.

1. It receives the current input.
2. It combines the current input with the previous hidden state.
3. It calculates a new hidden state.
4. It passes this hidden state to the next time step.
5. The same recurrent weights are reused at each time step.

RNN formula:

$h_t = \tanh(W_{xh} x_t + W_{hh} h_{(t-1)} + b_h)$

Where:

- h_t = Current hidden state
- x_t = Current input
- $h_{(t-1)}$ = Previous hidden state
- W_{xh} = Input-to-hidden weights
- W_{hh} = Recurrent weights
- b_h = Bias
- tanh = Activation function

Simple RNN flow

Input at time step 1 → Hidden state h_1

Input at time step 2 + h_1 → Hidden state h_2

Input at time step 3 + h_2 → Hidden state h_3

The hidden state carries information from earlier steps to later steps.

Advantages of RNN

- Processes sequential data.
- Maintains contextual information through its hidden state.
- Reuses the same weights across time steps.
- Can process sequences of varying lengths.

Limitations of RNN

1. Vanishing Gradient Problem

During backpropagation through time, gradients can become extremely small. This makes it difficult for a basic RNN to learn long-term dependencies.

2. Exploding Gradient Problem

Gradients can also become excessively large, making training unstable.

3. Limited Long-Term Memory

A basic RNN can struggle to preserve useful information across long sequences.

Applications of RNN

- Text processing
- Language modeling
- Speech recognition
- Time-series forecasting
- Sequential data analysis

---

2. Long Short-Term Memory (LSTM)

What is LSTM?

Long Short-Term Memory (LSTM) is a specialized type of RNN designed to learn long-term dependencies more effectively.

It was developed to address the difficulty basic RNNs have in preserving information over long sequences, particularly because of the vanishing gradient problem.

LSTM uses a cell state and three gates to control the flow of information.

Why do we need LSTM?

Suppose a model reads a long sentence. Information from an earlier word might be important for understanding a word that appears much later.

A basic RNN may struggle to preserve this information during training.

LSTM provides a cell state and gating mechanisms that help retain important information and discard irrelevant information.

Main Components of LSTM

LSTM has three gates and a cell state.

1. Forget Gate

The forget gate decides which information from the previous cell state should be retained or discarded.

Formula:

$f_t = \sigma(W_f [h_{(t-1)}, x_t] + b_f)$

Here, σ is the sigmoid function, which produces values between 0 and 1.

- A value close to 0 means forget most of that information.
- A value close to 1 means retain most of that information.

2. Input Gate

The input gate decides which new information should be stored in the cell state.

It uses two calculations:

Input gate:

$i_t = \sigma(W_i [h_{(t-1)}, x_t] + b_i)$

Candidate cell information:

$\tilde{C}_t = \tanh(W_C [h_{(t-1)}, x_t] + b_C)$

The input gate controls how much of the candidate information is added to the cell state.

3. Cell State

The cell state carries information through the sequence and helps the network preserve useful information over longer periods.

The cell state is updated using the forget gate and input gate:

$C_t = f_t \odot C_{(t-1)} + i_t \odot \tilde{C}_t$

Where:

- C_t = Current cell state
- $C_{(t-1)}$ = Previous cell state
- f_t = Forget gate output
- i_t = Input gate output
- $\tilde{C}_t$ = Candidate cell information
- $\odot$ = Element-wise multiplication

4. Output Gate

The output gate controls how much information from the current cell state contributes to the hidden state.

Output gate formula:

$o_t = \sigma(W_o [h_{(t-1)}, x_t] + b_o)$

Hidden state formula:

$h_t = o_t \odot \tanh(C_t)$

The hidden state can then be passed to the next time step or used to produce a prediction.

How LSTM Works: Step by Step

1. The forget gate decides what information to retain from the previous cell state.
2. The input gate selects new information to store.

3. The cell state is updated with retained and selected information.
4. The output gate determines the information exposed through the hidden state.
5. The updated hidden state and cell state are passed to the next time step.

Advantages of LSTM

- Learns long-term dependencies more effectively than a basic RNN.
- Helps reduce the vanishing gradient problem.
- Controls information flow through its gates.
- Can be used for sequential data with long-range relationships.

Limitations of LSTM

- More complex than a basic RNN.
- Has more parameters and computational overhead.
- Sequential processing can limit training parallelism.
- Does not completely eliminate all gradient-related problems.

Applications of LSTM

- Text generation
- Language modeling
- Speech recognition
- Time-series forecasting
- Sentiment analysis
- Sequence classification

---

3. Difference Between RNN and LSTM

Feature	RNN	LSTM
Architecture	Recurrent neural network	Specialized RNN
Memory mechanism	Hidden state	Hidden state and cell state
Gates	No standard LSTM-style gates	Forget, input and output gates
Long-term dependencies	Often difficult to learn	Generally better at learning them
Vanishing gradients	More susceptible	Helps mitigate the problem
Complexity	Simpler	More complex
Computational cost	Generally lower	Generally higher

---

4. Important Concepts to Remember

- RNN stands for Recurrent Neural Network.
- LSTM stands for Long Short-Term Memory.
- RNN processes sequential data using a hidden state.
- RNN reuses the same recurrent weights at each time step.
- A basic RNN can suffer from vanishing and exploding gradients.
- LSTM uses three gates: forget gate, input gate and output gate.
- The cell state helps carry information across time steps.
- The hidden state represents information used for the current output and subsequent processing.

- LSTM helps reduce the vanishing gradient problem but does not guarantee its complete elimination.
- GRU (Gated Recurrent Unit) is another gated recurrent architecture.
- Modern GPT-style LLMs use Transformer architectures rather than conventional RNNs.

5. Learning Path After RNN and LSTM

1. Gated Recurrent Unit (GRU)
2. Attention Mechanism
3. Transformer Architecture
4. Tokenization and Token IDs
5. Embeddings
6. Self-Attention and Multi-Head Attention
7. GPT and Decoder-Only Transformers
8. Large Language Models (LLMs)
9. Generative AI Applications
10. Retrieval-Augmented Generation (RAG)